EXP 2

```
aiml-ds@aimlds-ThinkCentre-M70s:~$ mkdir prac1
aiml-ds@aimlds-ThinkCentre-M70s:~$ mkdir prac2
aiml-ds@aimlds-ThinkCentre-M70s:~$ cd prac1
aiml-ds@aimlds-ThinkCentre-M70s:~/prac1$ touch exp1
aiml-ds@aimlds-ThinkCentre-M70s:~/prac1$ echo "linux
> iot
> dbms
> ct"> exp1
aiml-ds@aimlds-ThinkCentre-M70s:~/prac1$ ls -l
total 4
-rw-rw-r-- 1 aiml-ds aiml-ds 18 Jan 27 16:25 exp1
aiml-ds@aimlds-ThinkCentre-M70s:~/prac1$ cat exp1
linux
iot
dbms
ct
aiml-ds@aimlds-ThinkCentre-M70s:~/prac1$ sort exp1
ct
dbms
iot
linux
aiml-ds@aimlds-ThinkCentre-M70s:~/prac1$ head -2 exp1
linux
iot
aiml-ds@aimlds-ThinkCentre-M70s:~/prac1$ tail -2 exp1
dbms
ct
aiml-ds@aimlds-ThinkCentre-M70s:~/prac1$ touch exp2
aiml-ds@aimlds-ThinkCentre-M70s:~/prac1$ echo "exp2"> exp2
aiml-ds@aimlds-ThinkCentre-M70s:~/prac1$ ls -l
total 8
-rw-rw-r-- 1 aiml-ds aiml-ds 18 Jan 27 16:25 exp1
-rw-rw-r-- 1 aiml-ds aiml-ds  5 Jan 27 16:27 exp2
aiml-ds@aimlds-ThinkCentre-M70s:~/prac1$ cat exp2
```

EXPERIMENT NO 02

```
aiml-ds@aimlds-ThinkCentre-M70s: ~
linux
iot
dbms
ct
aiml-ds@aimlds-ThinkCentre-M70s:~/prac1$ chmod 755 exp2
aiml-ds@aimlds-ThinkCentre-M70s:~/prac1$ ls -l
total 4
-rwxr-xr-x 1 aiml-ds aiml-ds 18 Jan 27 16:25 exp2
aiml-ds@aimlds-ThinkCentre-M70s:~/prac1$ ps
    PID TTY          TIME CMD
   5722 pts/0    00:00:00 bash
   5819 pts/0    00:00:00 ps
aiml-ds@aimlds-ThinkCentre-M70s:~/prac1$ top

top - 16:29:47 up  1:27,  1 user,  load average: 0.08, 0.11, 0.10
Tasks: 298 total,   1 running, 297 sleeping,   0 stopped,   0 zombie
%Cpu(s):  1.0 us,  0.5 sy,  0.0 ni, 98.0 id,  0.5 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   7683.1 total,   3189.8 free,   1912.6 used,   2580.8 buff/cache
MiB Swap:   2048.0 total,   2048.0 free,      0.0 used.   4963.1 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   5820 aiml-ds   20   0   13224   4224   3328 R   6.2   0.1   0:00.01 top
      1 root      20   0  166844  11480   8152 S   0.0   0.1   0:01.18 systemd
      2 root      20   0       0      0      0 S   0.0   0.0   0:00.00 kthreadd
      3 root      20   0       0      0      0 S   0.0   0.0   0:00.00 pool_workqueue_relea
      4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_g
      5 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_p
      6 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-slub_
      7 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-netns
                                                 0.0   0.0   0:00.00 kworker/0:0H-events
```

EXP 3

AGE NO
ATE :

Exp 3

```
ls@ds-ThinkCentre-M70s:~/exp3$ mkdir exp3
ls@ds-ThinkCentre-M70s:~/exp3$ cd exp3
ls@ds-ThinkCentre-M70s:~/exp3/exp3$ nano file1.txt
ls@ds-ThinkCentre-M70s:~/exp3/exp3$ cat file1.txt
this is practical 3

ls@ds-ThinkCentre-M70s:~/exp3/exp3$ cp file1.txt file2.txt
ls@ds-ThinkCentre-M70s:~/exp3/exp3$ cat file2.txt
this is practical 3
```

Output -

```
ds@ds-ThinkCentre-M70s:~/exp3/exp3$ strace cp file1.txt file2.txt
execve("/usr/bin/cp", ["cp", "file1.txt", "file2.txt"], 0x7ffeff52d1a0 /* 46 vars */) = 0
brk(NULL)                               = 0x56c38cd48000
arch_prctl(0x3001 /* ARCH_??? */, 0x7fffa8548600) = -1 EINVAL (Invalid argument)
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x792b8e17d000
```

EXP 4

1)Display OS version, release number, kernel version

CODE:

```
 #!/bin/bash
echo "OS Version & Release:"
lsb_release -a
echo
echo "Kernel Version:"
uname -r
```

START :

```
 nano info .sh
 Chmod +x info.sh
./info.sh
```

2) Display top 10 processes in descending order

CODE:

```
#!/bin/bash
echo "Top 10 Processes by CPU Usage:"
ps -eo pid,ppid,cmd,%cpu --sort=-%cpu | head -11
```

START:

```
nano cpu.sh
chmod +x cpu.sh
./cpu.sh
```

3) Display processes with highest memory usage.

CODE:

```
#!/bin/bash
echo "Top Processes by Memory Usage:"
ps -eo pid,ppid,cmd,%cpu --sort=-%cpu | head -11
```

START:

```
nano mem.sh
```

chmod +x mem.sh

./mem.sh

4) Display current logged in user and log name.

CODE:

```
#!/bin/bash
echo "Current User:"
whoami
echo "Login Name:"
logname
```

START:

Nano user.sh

Chmod +x user.sh

./user.sh

5) e) Display current shell, home directory, operating system type, current path setting, current working directory.

CODE:

```
#!/bin/bash
echo "Current Shell:"
echo $SHELL

echo "Home Directory:"
echo $HOME

echo "Operating System Type:"
uname

echo "Current Path Setting:"
echo $PATH
```

echo "Current Working Directory:"

pwd

START:

Nano system.sh

Chmod +x system.sh

./system.sh

Nano pract.c

Gcc pract.c -o pract

./pract

Same for exp5 -exp 10

EXP 5

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
int main()
{
    pid_t pid;

    pid = fork(); // Create child process

    if (pid < 0)
    {
        printf("Fork failed\n");
    }
    else if (pid == 0)
    {
        // Child process
        printf("Child Process\n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID : %d\n", getppid());
    }
    else
    {
        // Parent process
        printf("Parent Process\n");
        printf("Parent PID : %d\n", getpid());
    }
    return 0;
}
```

EXP 6

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
int main()
{
   pid_t pid;

   pid = fork(); // Create child process

   if (pid < 0)
   {
      printf("Fork failed\n");
   }
   else if (pid == 0)
   {
      printf("Child Process\n");
      printf("Child PID : %d\n", getpid());
      printf("Parent PID : %d\n", getppid());
   }
   else
   {
      // Parent process
      printf("Parent Process\n");
      printf("Parent PID : %d\n", getpid());
   }
  return 0;
}
```

EXP 7

```
#include <stdio.h>
#include <semaphore.h>

sem_t mutex, full, empty;
int x = 0;

void producer()
{
    sem_wait(&empty);
    sem_wait(&mutex);

    x++;
    printf("Producer produces item %d\n", x);

    sem_post(&mutex);
    sem_post(&full);
}

void consumer()
{
    sem_wait(&full);
    sem_wait(&mutex);

    printf("Consumer consumes item %d\n", x);
    x--;

    sem_post(&mutex);
    sem_post(&empty);
```

```
}

int main()
{
    sem_init(&mutex, 0, 1);
    sem_init(&empty, 0, 1);
    sem_init(&full, 0, 0);

    producer();
    consumer();

    return 0;
}
```

EXP 8

```
#include <stdio.h>

int main()
{
    int block1[5] = {100, 500, 200, 300, 600};
    int block2[5] = {100, 500, 200, 300, 600};
    int block3[5] = {100, 500, 200, 300, 600};
    int process[4] = {212, 417, 112, 426};
    int i, j;

    // First Fit
    printf("First Fit:\n");
    for(i = 0; i < 4; i++)
    {
        for(j = 0; j < 5; j++)
        {
            if(block1[j] >= process[i])
            {
                printf("Process %d allocated to Block %d\n", i+1, j+1);
                block1[j] -= process[i];
                break;
            }
        }
    }

    // Best Fit
    printf("\nBest Fit:\n");
    for(i = 0; i < 4; i++)
```

```
{
    int best = -1;
    for(j = 0; j < 5; j++)
    {
        if(block2[j] >= process[i])
        {
            if(best == -1 || block2[j] < block2[best])
                best = j;
        }
    }
    if(best != -1)
    {
        printf("Process %d allocated to Block %d\n", i+1, best+1);
        block2[best] -= process[i];
    }
}

// Worst Fit
printf("\nWorst Fit:\n");
for(i = 0; i < 4; i++)
{
    int worst = -1;
    for(j = 0; j < 5; j++)
    {
        if(block3[j] >= process[i])
        {
            if(worst == -1 || block3[j] > block3[worst])
                worst = j;
        }
```

```
        }
        if(worst != -1)
        {
            printf("Process %d allocated to Block %d\n", i+1, worst+1);
            block3[worst] -= process[i];
        }
    }

    return 0;
}
```

EXP 9

```
#include <stdio.h>

int main()
{
    int pages[] = {7, 0, 1, 2, 0, 3, 0, 4};
    int frames[3];
    int i, j, k, flag, faults = 0;

    // Initialize frames
    for(i = 0; i < 3; i++)
        frames[i] = -1;

    printf("FIFO Page Replacement\n");
    int index = 0;

    for(i = 0; i < 8; i++)
    {
        flag = 0;

        for(j = 0; j < 3; j++)
        {
            if(frames[j] == pages[i])
            {
                flag = 1;
                break;
            }
        }
```

```
    if(flag == 0)
    {
        frames[index] = pages[i];
        index = (index + 1) % 3;
        faults++;
    }

    printf("Page %d -> ", pages[i]);
    for(j = 0; j < 3; j++)
        printf("%d ", frames[j]);
    printf("\n");
}

printf("Total Page Faults = %d\n\n", faults);

// LRU
printf("LRU Page Replacement\n");
faults = 0;

int time[3], counter = 0;

for(i = 0; i < 3; i++)
{
    frames[i] = -1;
    time[i] = 0;
}

for(i = 0; i < 8; i++)
{
```

```
flag = 0;

for(j = 0; j < 3; j++)
{
    if(frames[j] == pages[i])
    {
        counter++;
        time[j] = counter;
        flag = 1;
        break;
    }
}

if(flag == 0)
{
    int min = 0;
    for(j = 1; j < 3; j++)
        if(time[j] < time[min])
            min = j;

    frames[min] = pages[i];
    counter++;
    time[min] = counter;
    faults++;
}

printf("Page %d -> ", pages[i]);
for(j = 0; j < 3; j++)
    printf("%d ", frames[j]);
```

```
    printf("\n");
}

printf("Total Page Faults = %d\n\n", faults);

// Optimal
printf("Optimal Page Replacement\n");
faults = 0;

for(i = 0; i < 3; i++)
    frames[i] = -1;

for(i = 0; i < 8; i++)
{
    flag = 0;

    for(j = 0; j < 3; j++)
    {
        if(frames[j] == pages[i])
        {
            flag = 1;
            break;
        }
    }

    if(flag == 0)
    {
        int pos = -1, farthest = i;
```

```
for(j = 0; j < 3; j++)
{
    int found = 0;

    for(k = i + 1; k < 8; k++)
    {
        if(frames[j] == pages[k])
        {
            if(k > farthest)
            {
                farthest = k;
                pos = j;
            }
            found = 1;
            break;
        }
    }

    if(!found)
    {
        pos = j;
        break;
    }
}

if(pos == -1)
    pos = 0;

frames[pos] = pages[i];
```

```
        faults++;
    }

    printf("Page %d -> ", pages[i]);
    for(j = 0; j < 3; j++)
        printf("%d ", frames[j]);
    printf("\n");
  }

  printf("Total Page Faults = %d\n", faults);

  return 0;
}
```

EXP 10

Program 1: FCFS Disk Scheduling

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
   int req[] = {98, 183, 37, 122, 14, 124, 65, 67};
   int head = 53;
   int seek = 0;

   for(int i = 0; i < 8; i++)
   {
      seek += abs(req[i] - head);
      head = req[i];
   }

   printf("Total Seek Time (FCFS) = %d", seek);
   return 0;
}
```

Program 2: SCAN Disk Scheduling

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
   int req[] = {65, 67, 98, 122, 124, 183, 37, 14};
   int head = 53;
   int seek = 0;
```

```
    int disk = 199;

    for(int i = 0; i < 6; i++)
    {
        seek += abs(req[i] - head);
        head = req[i];
    }

    seek += abs(disk - head);
    head = disk;

    for(int i = 6; i < 8; i++)
    {
        seek += abs(req[i] - head);
        head = req[i];
    }

    printf("Total Seek Time (SCAN) = %d", seek);
    return 0;
}
```

Program 3: C-SCAN Disk Scheduling

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int req[] = {65, 67, 98, 122, 124, 183, 14, 37};
    int head = 53;
    int seek = 0;
```

```
    int disk = 199;

    for(int i = 0; i < 6; i++)
    {
        seek += abs(req[i] - head);
        head = req[i];
    }

    seek += abs(disk - head);

    // Jump to beginning (C-SCAN)
    head = 0;

    for(int i = 6; i < 8; i++)
    {
        seek += abs(req[i] - head);
        head = req[i];
    }

    printf("Total Seek Time (C-SCAN) = %d", seek);
    return 0;
}
```